

Exercícios sobre Struct's

1. Criando uma Estrutura Simples

Crie uma struct chamada Pessoa, que contém os seguintes campos:

- Nome (string de até 50 caracteres)
- Idade (inteiro)
- Altura (float)

No programa principal, declare uma variável do tipo Pessoa, preencha os dados do usuário via teclado e exiba as informações.

2. Vetor de Structs

Crie um vetor de 5 elementos da struct Aluno, que contém:

- Nome (string de até 50 caracteres)
- Matrícula (inteiro)
- Nota (float)

Solicite ao usuário que insira os dados dos 5 alunos e depois exiba a média das notas.

3. Structs e Funções

Crie uma struct chamada Produto, que contém:

- Nome do produto (string de até 30 caracteres)
- Código (inteiro)
- Preço (float)

Crie uma função que recebe um Produto como parâmetro e exibe seus dados formatados. No main(), instancie um produto, peça ao usuário que insira seus valores e chame a função para exibir os dados.

4. Structs e Ponteiros

Crie uma struct chamada Carro, com os seguintes campos:

- Modelo (string de até 30 caracteres)

- Ano (inteiro)
- Preço (float)

No programa, declare um ponteiro para uma struct Carro e use malloc() para alocar memória dinamicamente. Preencha os dados e exiba as informações antes de liberar a memória com free().

5. Registro de Funcionários

Crie um programa que gerencie um pequeno sistema de funcionários. Cada funcionário deve ter:

- Nome (string de até 50 caracteres)
- Cargo (string de até 30 caracteres)
- Salário (float)

Permita ao usuário inserir os dados de N funcionários (onde N é informado pelo usuário) e exiba uma lista com todos os funcionários cadastrados.

6. Cadastro de Livros

Crie uma struct chamada Livro, que contenha:

- Título (string de até 50 caracteres)
- Autor (string de até 50 caracteres)
- Ano de publicação (inteiro)

Crie uma função que recebe um vetor de Livro e exibe os livros publicados após um determinado ano (definido pelo usuário).

7. Agenda de Contatos

Crie uma agenda telefônica usando structs, onde cada contato possui:

- Nome (string de até 50 caracteres)
- Telefone (string de até 15 caracteres)

O programa deve permitir adicionar contatos e buscar um contato pelo nome.

8. Cálculo de Área e Perímetro

Crie uma struct chamada Retangulo, que contém:

- Base (float)
- Altura (float)

Crie funções para calcular a área e o perímetro do retângulo. No main(), permita que o usuário insira os valores e exiba os cálculos.

9. Registro de Pacientes

Crie uma struct chamada Paciente, contendo:

- Nome (string de até 50 caracteres)
- Idade (inteiro)
- Diagnóstico (string de até 100 caracteres)

Crie uma função que recebe um vetor de Paciente e exibe todos os pacientes acima de 60 anos.

10. Sistema Bancário Simples

Crie uma struct chamada ContaBancaria, contendo:

- Nome do titular (string de até 50 caracteres)
- Número da conta (inteiro)
- Saldo (float)

Crie funções para:

- Depositar um valor na conta
- Sacar um valor da conta (apenas se houver saldo suficiente)
- Exibir o saldo

No main(), crie uma conta bancária e permita que o usuário realize operações.

Exercícios sobre Algoritmos Recursivos x Iterativos

Exercícios de Recursividade

1. Número de Dígitos de um Inteiro

Implemente uma função recursiva que recebe um número inteiro n e retorna a quantidade de dígitos que ele possui.

Exemplo:

- `contar_digitos(12345) → 5`
 - `contar_digitos(7) → 1`
-

2. Maior Elemento em um Vetor

Crie uma função recursiva que recebe um vetor de inteiros e seu tamanho n e retorna o maior valor presente no vetor.

Exemplo:

```
int maior_elemento(int vet[], int n);
```

3. Soma dos Dígitos de um Número

Implemente uma função recursiva que recebe um número inteiro n e retorna a soma de seus dígitos.

Exemplo: `soma_digitos(1234) → 1 + 2 + 3 + 4 = 10`.

4. Produto de Dois Números Usando Somas Sucessivas

Implemente uma função recursiva que calcula o produto de dois números inteiros a e b apenas usando somas sucessivas (sem usar $*$).

Exemplo:

- `produto(5, 3) → 5 + 5 + 5 = 15`
 - `produto(7, 2) → 7 + 7 = 14`
-

5. Contagem Regressiva

Crie uma função recursiva que recebe um número n e imprime uma contagem regressiva até 0.

Exemplo: `contagem_regressiva(5)` → 5, 4, 3, 2, 1, 0.

6. Verificar se uma String é Palíndromo

Implemente uma função recursiva que recebe uma string e retorna 1 se for um palíndromo e 0 caso contrário.

Exemplo:

- `palindromo("arara")` → 1
 - `palindromo("casa")` → 0
-

Transformação de Recursivos para Iterativos

7. Transformar Contagem Regressiva em Iterativa

Pegue a função recursiva do exercício 5 e implemente uma versão iterativa usando `for` ou `while`.

8. Transformar Maior Elemento em Iterativo

Pegue a função recursiva do exercício 2 e implemente uma versão iterativa para encontrar o maior elemento do vetor.

9. Transformar Soma dos Dígitos em Iterativo

Pegue a função do exercício 3 e implemente uma versão iterativa que calcula a soma dos dígitos.

10. Transformar Produto de Dois Números em Iterativo

Pegue a função do exercício 4 e implemente uma versão iterativa que calcula o produto usando apenas somas sucessivas.

Exercícios de Listas, Pilhas e Filas

Exercícios sobre Listas (4 exercícios)

1. Implementação de uma Lista Encadeada Simples

Crie um programa que implemente uma lista encadeada simples com as seguintes operações:

- **Inserir um elemento no início.**
 - **Inserir um elemento no final.**
 - **Remover um elemento específico.**
 - **Exibir todos os elementos da lista.**
-

2. Contagem de Elementos na Lista

Modifique o código do exercício 1 e implemente uma função para contar quantos elementos existem na lista.

3. Inserção Ordenada na Lista Encadeada

Altere a implementação da lista encadeada para que os elementos sejam inseridos em ordem crescente automaticamente.

4. Reverter uma Lista Encadeada

Implemente uma função que inverte a ordem dos elementos de uma lista encadeada sem criar uma nova lista.

Exercícios sobre Pilhas (4 exercícios)

5. Implementação de uma Pilha com Vetor

Implemente uma pilha usando um vetor e forneça as seguintes operações:

- **push: inserir um elemento no topo.**
- **pop: remover o elemento do topo.**
- **top: exibir o elemento do topo.**

- **isEmpty:** verificar se a pilha está vazia.
-

6. Verificar se uma Expressão com Parênteses Está Correta

Crie uma função que recebe uma string contendo (e) e verifica se os parênteses estão balanceados.

Exemplo:

entrada: "(()())" → saída: Válido

entrada: "()" → saída: Inválido

7. Inverter uma String Usando uma Pilha

Implemente um programa que use uma pilha para inverter uma string.

8. Converter um Número Decimal para Binário Usando Pilha

Crie um programa que converta um número decimal em binário usando uma pilha.

Exemplo:

entrada: 10 → saída: 1010

entrada: 25 → saída: 11001

Exercícios sobre Filas (4 exercícios)

9. Implementação de uma Fila com Vetor

Implemente uma fila usando um vetor e forneça as seguintes operações:

- **enqueue:** inserir um elemento no final da fila.
 - **dequeue:** remover um elemento do início da fila.
 - **front:** exibir o primeiro elemento.
 - **isEmpty:** verificar se a fila está vazia.
-

10. Fila Circular

Implemente uma fila circular usando um vetor para evitar desperdício de espaço quando houver remoção de elementos.

11. Simular Atendimento em um Banco Usando Fila

Crie um programa que simule uma fila de atendimento bancário, aonde os clientes chegam e são atendidos na ordem de chegada.

12. Inverter uma Fila Usando uma Pilha

Implemente uma função que recebe uma fila e inverte sua ordem usando uma pilha como estrutura auxiliar.